```
describe "task completion" do
  let(:task) {tasks(:incomplete)}
  before(:example) { task.mark_complete }
  specify { expect(task).to be_complete }
  specify { expect(task).to be_blocked }
  specify { expect(task.end_date).to eq(Date.today.to_s(:db)) }
  specify { expect(task.most_recent_log.end_state).to eq("completed") }
end
```

You've already seen this pattern a bit, where the variable is defined in a let method, with some additional configuration happening in a before method.

The tradeoff is pretty plain: The one-assertion-per-test style has the advantage that each assertion can fail independently—when all the assertions are in a single test, the test bails on the first failure. In the all-in-one test, if expect(task).to be_complete fails, you won't even get to the check for expect(task).to be_blocked. If all the assertions are in separate tests, everything runs independently, but it's harder to determine how tests are related. There are two significant downsides to the one-assertion style. First, there can be a significant speed difference since the one-assertion-per-test version will run the common setup multiple times, and second, the one-assertion style can become difficult to read, especially if the setup and test wind up with some distance between them.

Often I compromise by making my first pass at TDD in the one-assertion-per-test style, which forces me to work in baby steps and gives me a more accurate picture of what tests are failing. When I'm confident in the correctness of the code, I consolidate related assertions, giving me the speed benefit moving forward. Another compromise is the use of compound RSpec matchers or the has_attributes matcher to create a single assertion out of what might otherwise be multiple assertions.

Recent versions of RSpec make this tradeoff significantly easier to manage with the :aggregate_failure metadata tag. When used, :aggregate_failure causes RSpec to no longer exit on the first failure in a spec, but to keep going and combine all the failures when reporting. (An error, like a missing method, still ends the spec.)

So, you can now have the speed or clarity advantage of a multiple-assertion test and still have the reporting fidelity of the single-assertion test:

```
it "marks a task complete", :aggregate_failures do
  task = tasks(:incomplete)
  task.mark_complete
  expect(task).to be_complete
  expect(task).to be_blocked
  expect(task.end_date).to eq(Date.today.to_s(:db))
  expect(task.most_recent_log.end_state).to eq("completed")
end
```